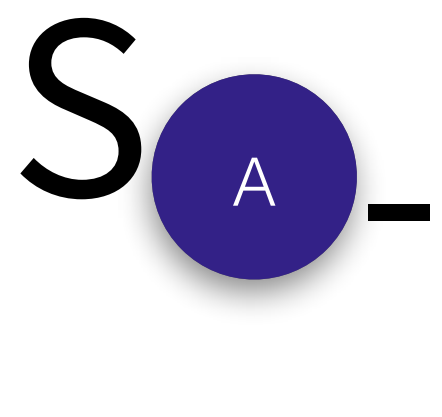
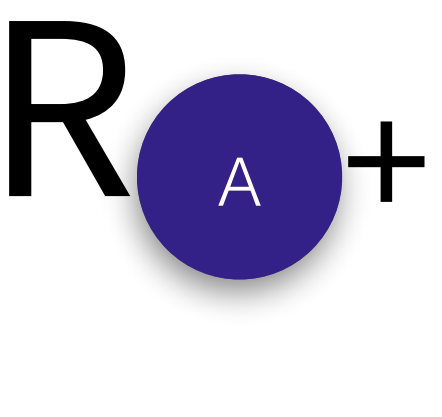
An Axiomatic Concurrency Model

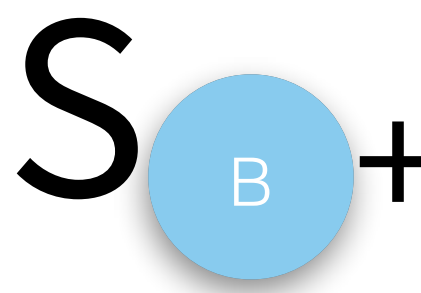
45

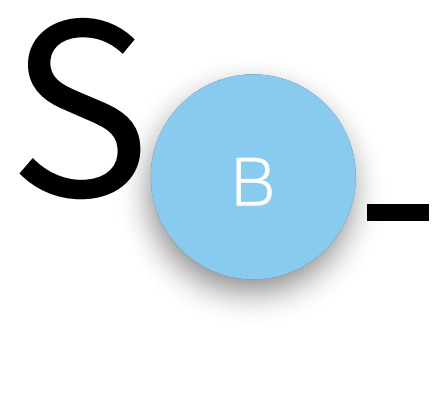

S
A
+

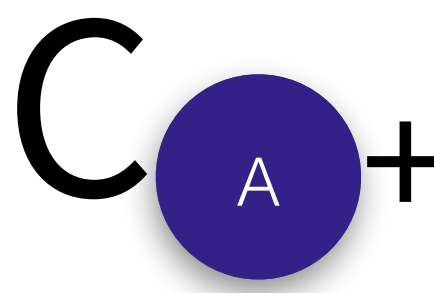

S
A

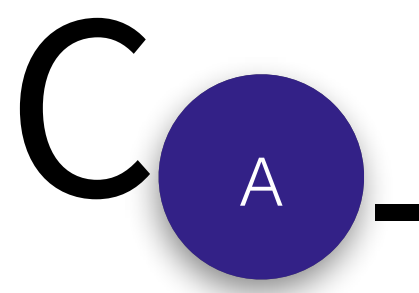

R
A
+

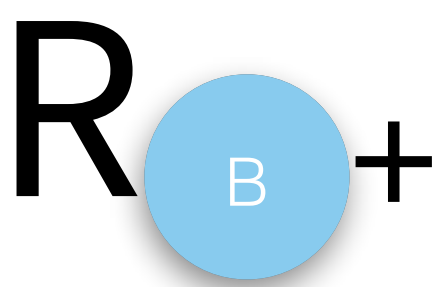

R
A
_

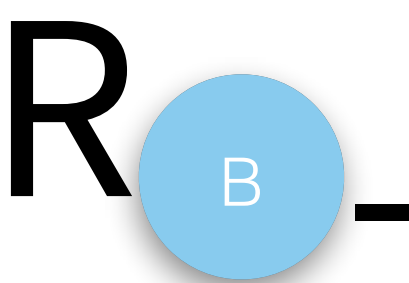

S
B
+

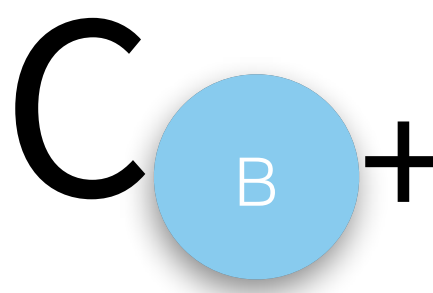

S
B

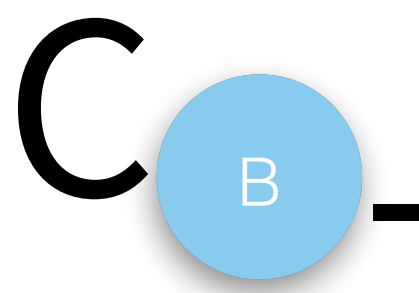

C
A
+


C
A
-


R
B
+


R
B
_


C
B
+


C
B


po

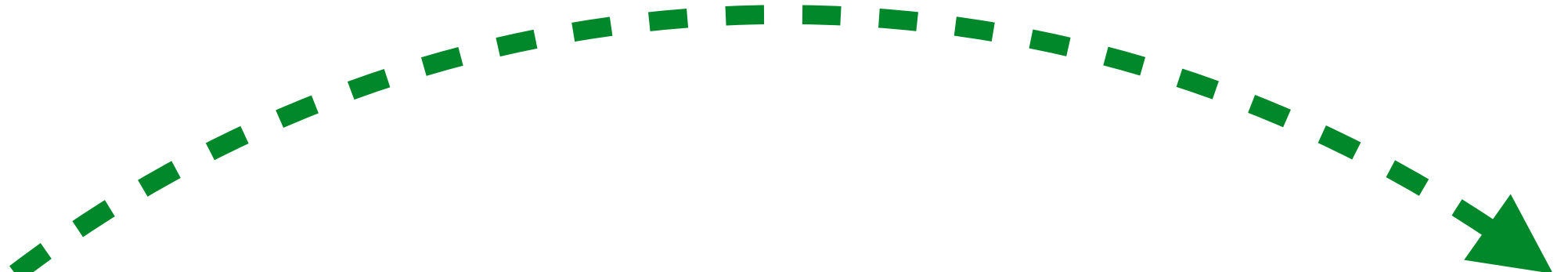

po

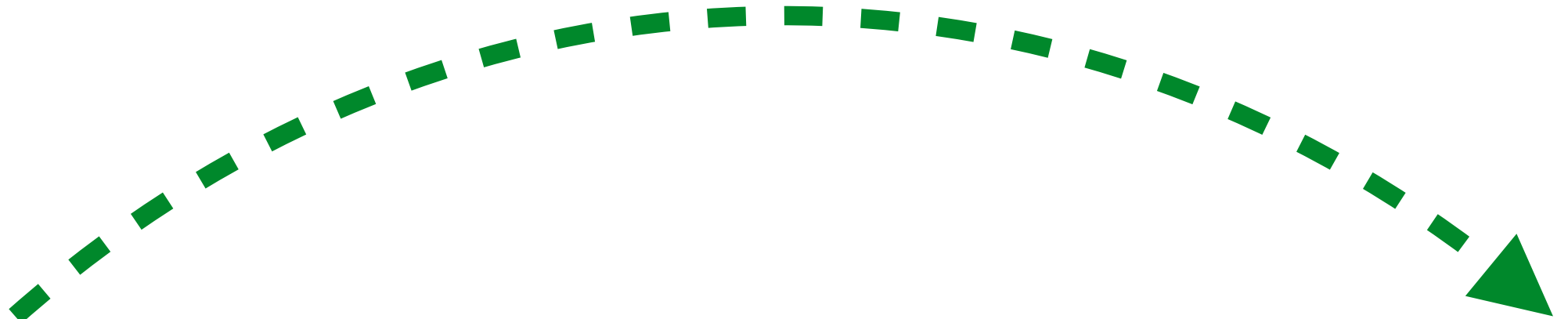

po

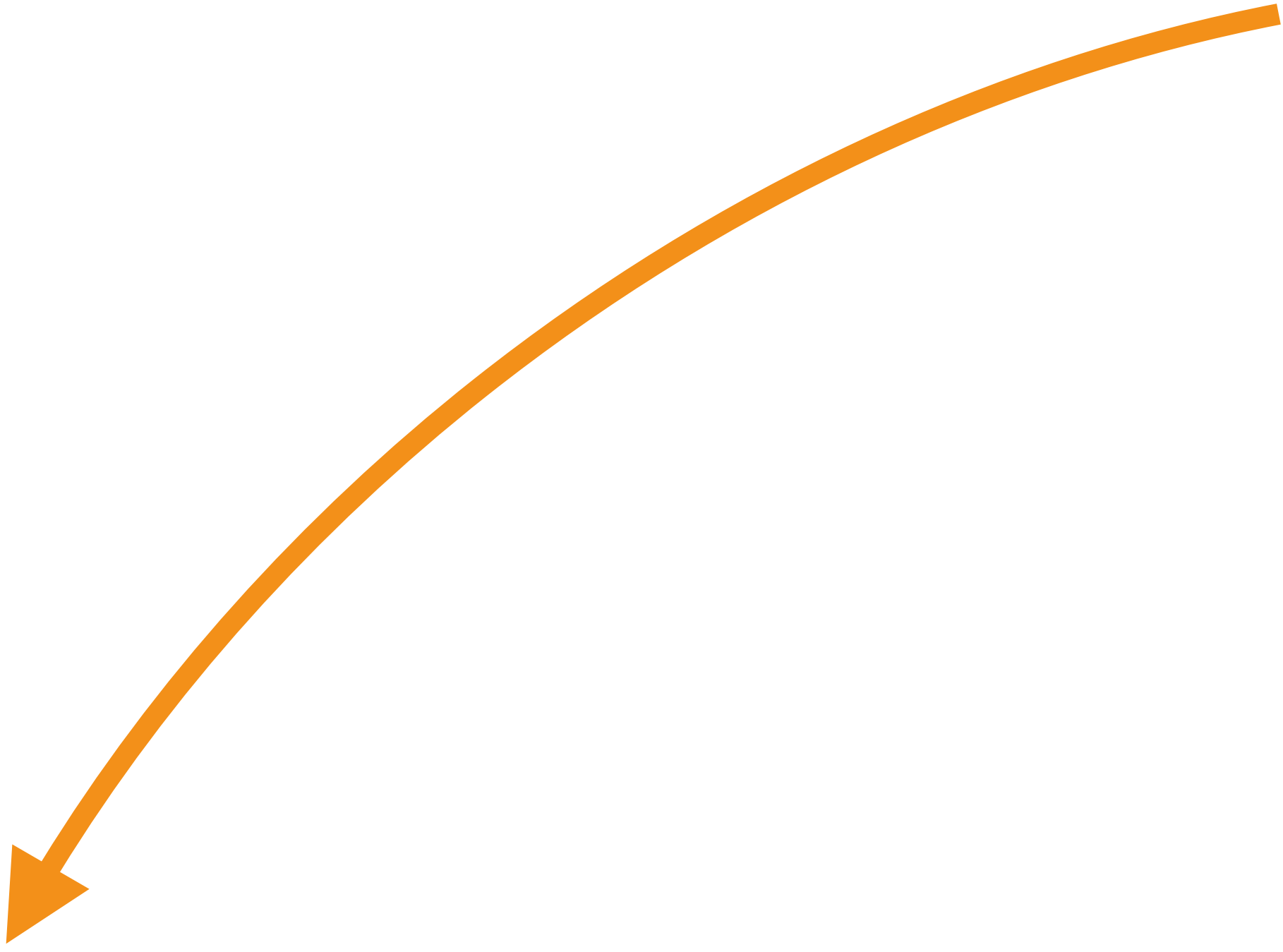

po


po

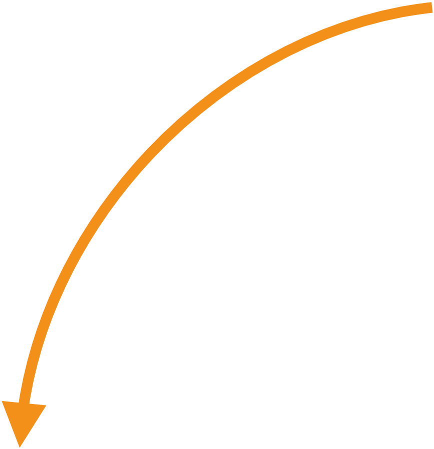

po



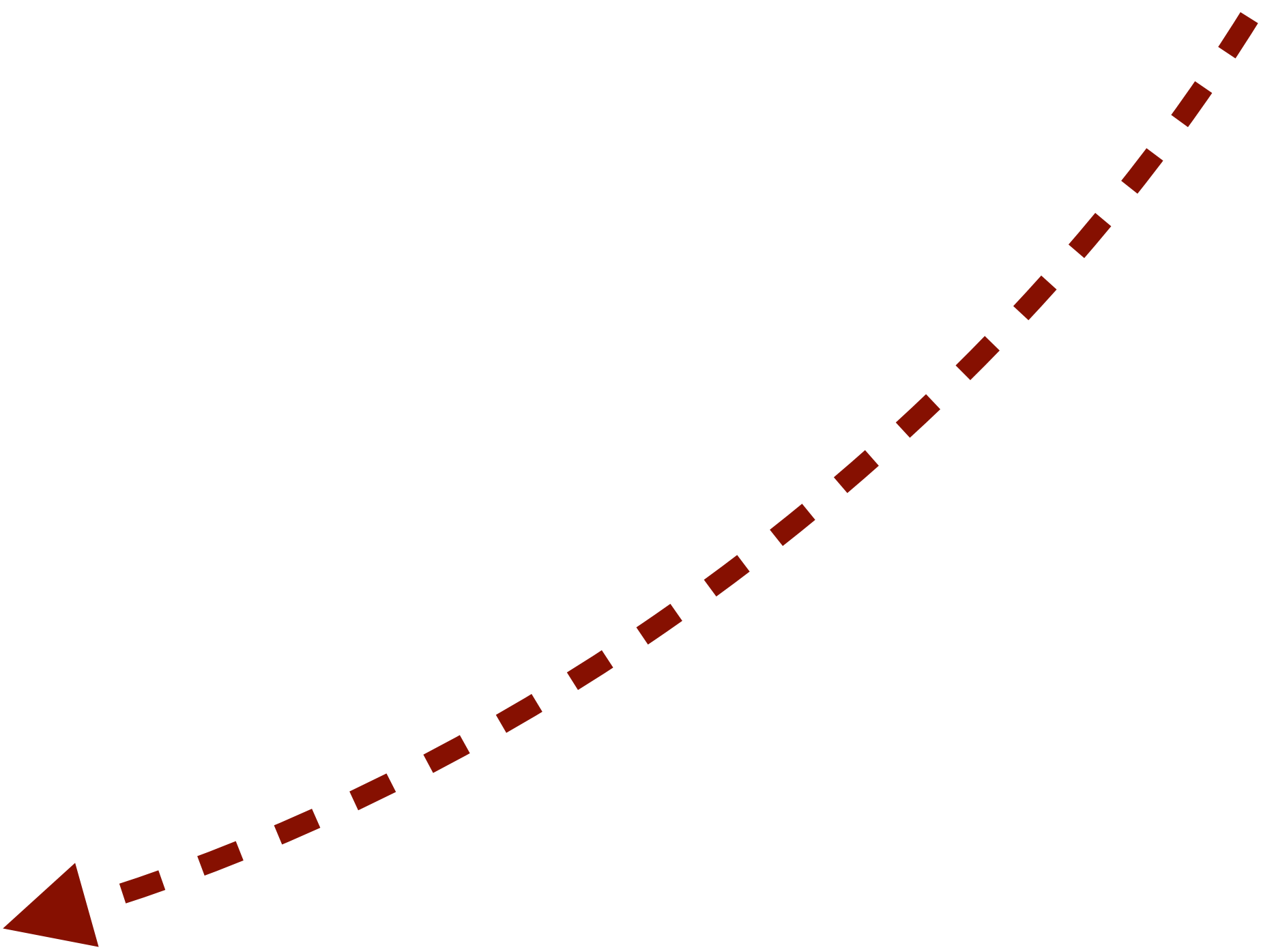




COsrc




COlog




h b


hb

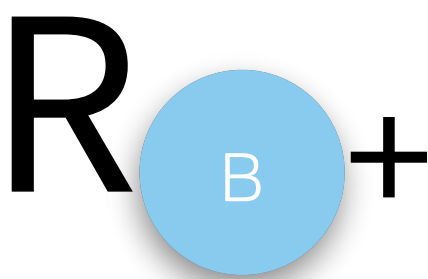
```
update(src, +100)
```

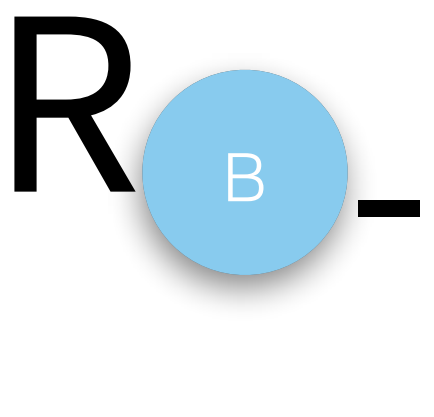
```
update(src, -100)
```

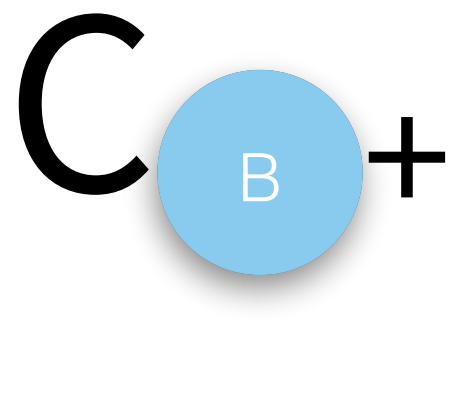
r

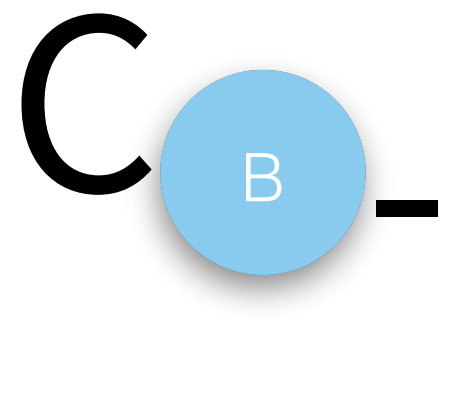
Valid executions are acyclic in the transitive closure of the union of relations

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```


R
B
+

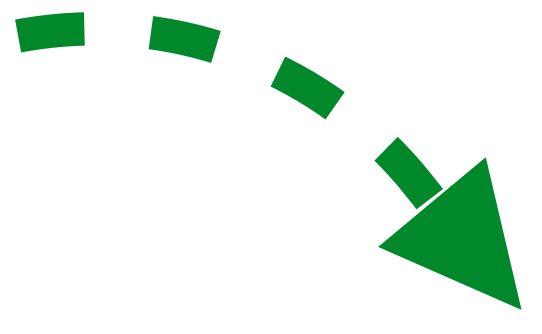

R
B
_


C
B
+

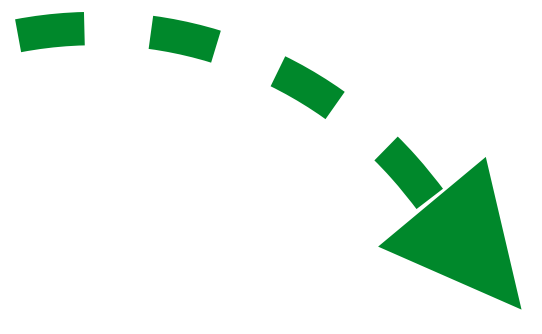

C
B

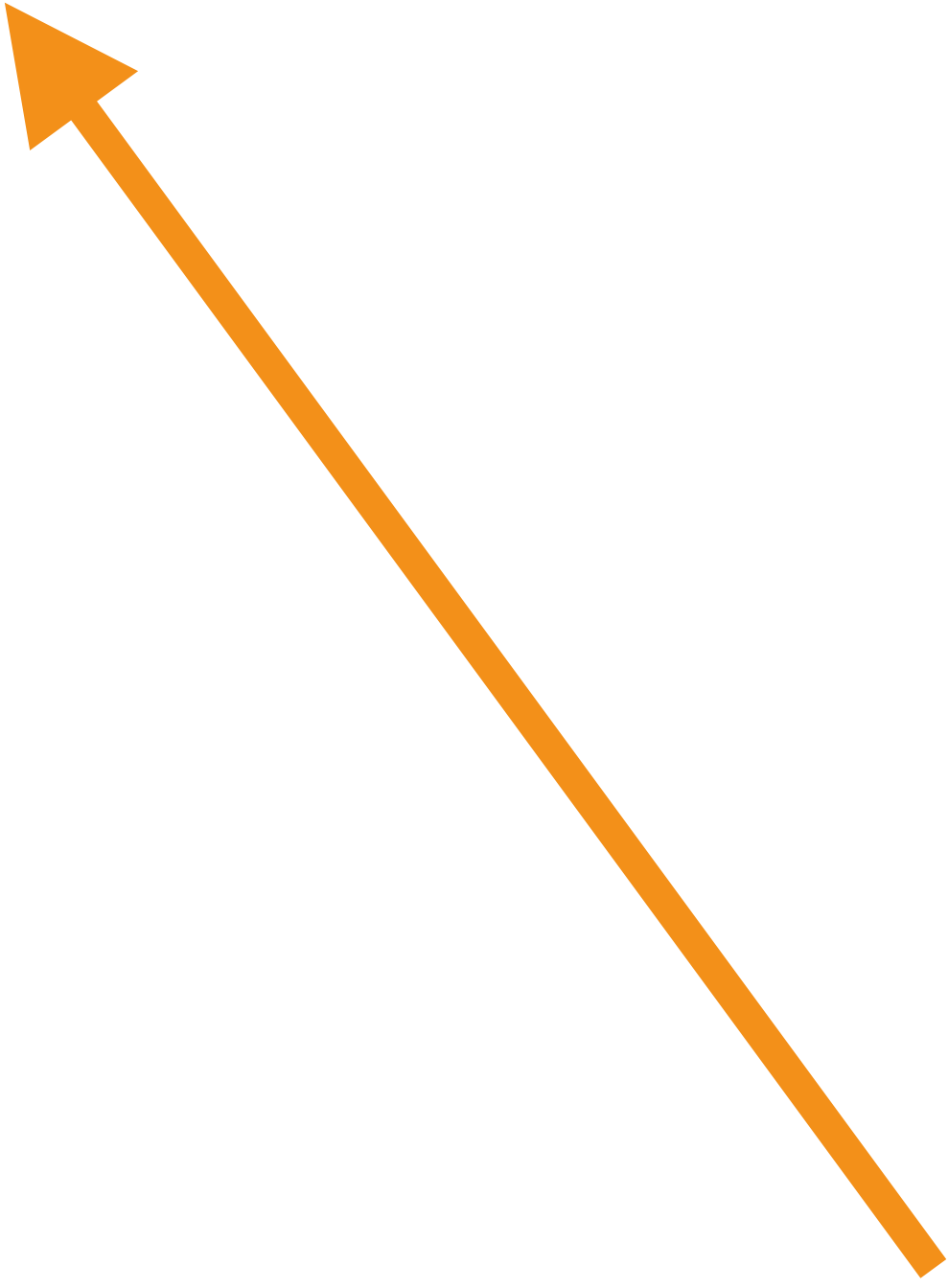
po


po




COlog






h b

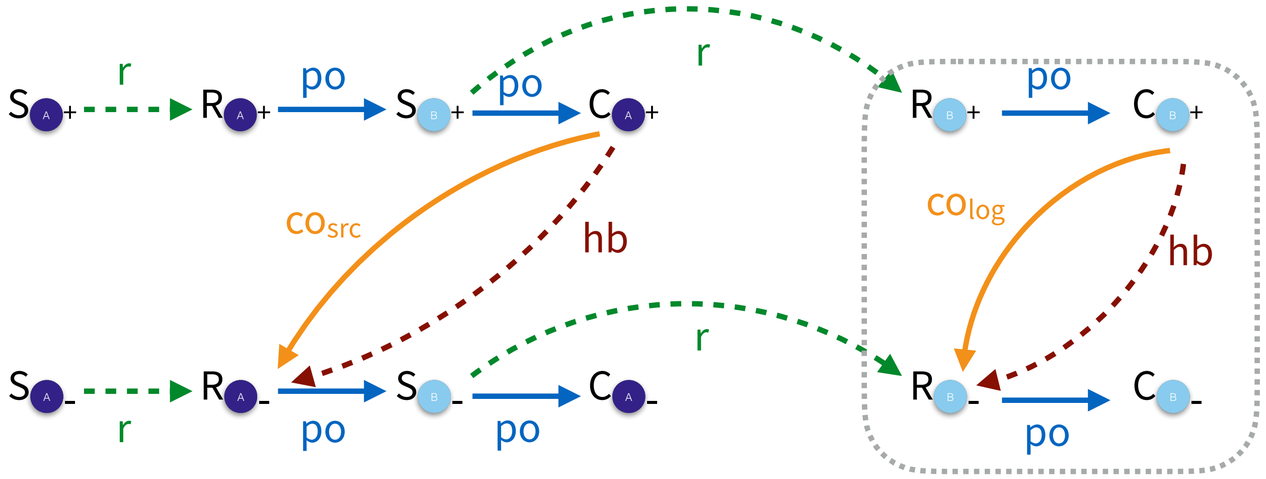
An Axiomatic Concurrency Model

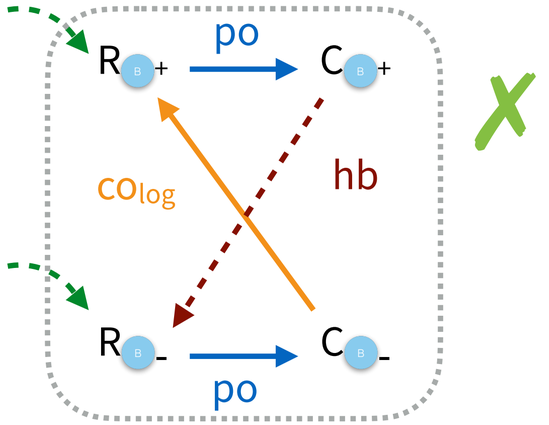
```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

```
update(src, +100)
```

```
update(src, -100)
```

Valid executions are acyclic in the transitive closure of the union of relations





What does this get us?

$$\frac{}{\textbf{when}(\overline{c})\{s_1\}; s_2 \hookrightarrow s_2 \mid (\overline{c}, s_1)}\ \textsc{Step}$$

$$\frac{s_1 \hookrightarrow s_2 \mid (\overline{c_3}, s_3) \qquad j \textbf{ fresh}}{\langle \overline{b_r}[(i, \overline{c_1}, s_1)], \overline{b_p}\rangle / H \rightsquigarrow \langle \overline{b_r}[(i, \overline{c_1}, s_2)], \overline{b_p} : (j, \overline{c_3}, s_3)\rangle / H[i \mathrel{+}= S_j]}\ \textsc{Spawn}$$

$$\frac{C = \{c \mid c \in \overline{c'} \wedge (\_, \overline{c'}, \_) \in \overline{b_r} \cup \overline{b_{p1}}\} \qquad C \cap \overline{c} = \emptyset}{\langle \overline{b_r}, \overline{b_{p1}} : (i, \overline{c}, s) : \overline{b_{p2}}\rangle / H \rightsquigarrow \langle \overline{b_r} : (i, \overline{c}, s), \overline{b_{p1}} : \overline{b_{p2}}\rangle / H[i \mathrel{+}= R_i][\overline{c} \mathrel{+}= R_i]}\ \textsc{Run}$$

$$\frac{}{\langle \overline{b_r}[(i, \overline{c}, \textbf{done})], \overline{b_p}\rangle / H \rightsquigarrow \langle \overline{b_r}[\epsilon], \overline{b_p}\rangle / H[i \mathrel{+}= C_i][\overline{c} \mathrel{+}= C_i]}\ \textsc{Complete}$$

Figure 9 Semantics of the core BoC calculus with histories.

$$\frac{}{i \vdash_b \epsilon}\ \textsc{WFB-Empty} \qquad \frac{}{i \vdash_b R_i : \overline{S} : (C_i?)}\ \textsc{WFB-NEmpty}$$

$$\frac{}{\vdash_c \epsilon}\ \textsc{WFC-Empty} \qquad \frac{}{\vdash_c R_i}\ \textsc{WFC-Single} \qquad \frac{\vdash_c \overline{E}}{\vdash_c R_i : C_i : \overline{e}}\ \textsc{WFC-Pair}$$

Figure 10 Well-formedness rules for behaviour and cown histories

4.3 Soundness of Core Calculus According to the Axiomatic Model

In the previous section, we showed that a well-formed history translates to a valid execution in the axiomatic model. In this section we show that the semantics is sound according the axiomatic model by proving that it always produces a well-formed history.

In addition to general well-formedness, the history of an execution is going to match the specific configuration that it was produced together with.

▶ **Theorem 15** (Preservation of well-formed and matching histories). *Starting from a well-formed and matching history, evaluation produces another well-formed and matching history:*

$$\tau \vdash H \wedge H; \tau \vdash cfg \wedge cfg/H \rightsquigarrow cfg'/H' \implies \exists \tau' \,.\, \tau' \vdash H' \wedge H'; \tau' \vdash cfg'$$

Proof. By cases on the evaluation relation. For each case we select a τ' that is τ extended so that the new event produced by that evaluation step is given a timestamp larger than any previous timestamp. Preservation of each of the two relations can then be proved separately. We refer to the mechanisation for details [7]. ◀

The operational semantics, and its implementations, are sound with respect to the axiomatic model